

Kuro

From Polling to Reactivity: An Event-Driven Blueprint for Maximizing Drone Farming Efficiency in 'Plant with Coding'

System Architecture and Bottleneck Analysis

The existing drone automation system for 'Plant with Coding' represents a modularized approach to a complex problem, utilizing five distinct code files to manage configuration, movement, farming operations, commands, and the main program flow [81](#). While this modularity provides a logical separation of concerns, its operational model is fundamentally constrained by a single-threaded, synchronous main loop that creates significant inefficiencies and bottlenecks, directly opposing the goal of minimizing in-game execution time. The core of the issue lies in the procedural execution pattern within `Automation5.lau`, which orchestrates all activities sequentially, leading to substantial periods of drone inactivity. The primary bottleneck is the fixed `task.wait(0.1)` call at the end of each main loop iteration [74](#). This command introduces an artificial delay of 100 milliseconds, during which the drone is idle regardless of whether it has completed its assigned tasks or if other conditions have changed. This polling-based approach is inherently inefficient, as it wastes processing cycles and extends the overall cycle time unnecessarily.

This rigid structure is compounded by the reliance on blocking operations throughout the system. For instance, the `moveTo` function in `Movement.laum` uses a busy-wait loop (`while drone.status() ~= Enum.DroneStatus.Sleep do task.wait(0.05) end`) to ensure the drone is stationary before proceeding [39](#). This method consumes CPU resources while waiting for the drone to become idle, contributing significantly to perceived lag and preventing any parallel processing. Similarly, the `waitForGrowth` function in `FarmingOperations.laum` employs a `task.wait(0.5)` loop to periodically check the growth percentage of a plant, further freezing the main thread until the specified condition is met [89](#). These patterns create a domino effect where the completion of one task is strictly dependent on the termination of the previous one, leaving the drone stationary for extended durations. The event-driven mechanisms provided by the game, such as `market.changedSeedStock:connect` and

`market.changedGearStock:connect`, are currently underutilized as isolated background processes [45](#) [46](#). They trigger their respective buying functions but lack a robust communication channel to signal completion back to the main loop, effectively making them fire-and-forget operations that do not integrate into the primary control flow.

Furthermore, the system exhibits signs of redundant state management. The `gardenScan` function in `FarmingOperations.lau` is designed to perform a full field scan, but this process can be forced by a chat command ("scanall") stored in the configuration [79](#). This suggests a need for a more dynamic and efficient state synchronization mechanism. The current reliance on flags like `buyingSeeds` and `buyingGears` serves as a simple lock to prevent re-entry into a long-running task, but these flags provide no information about the task's progress or outcome [41](#). The main loop cannot intelligently decide when to proceed because it lacks feedback on what was accomplished during the background purchase operation. This results in a passive waiting state, where the drone remains idle until the next scheduled action, such as the periodic auto-sell every 180 seconds, is triggered [56](#). The combination of these factors—a polling main loop, blocking I/O operations, and disconnected event handlers—creates a system architecture that is highly susceptible to performance degradation and fails to leverage Lau's inherent event-driven nature to its fullest potential. The subsequent analysis will focus on transforming this architecture to eliminate these bottlenecks and pave the way for a more responsive and efficient farming automation system.

The Central Role of Cache for Inter-Process Communication

The introduction of the `Cache` library is the most critical element for overcoming the architectural limitations of the current system. It provides a volatile, in-session memory space that allows for fast data exchange between different parts of the script, including the main loop and asynchronous event handlers [75](#). This capability is fundamental to establishing a reliable form of Inter-Process Communication (IPC) within the Lau environment, enabling a shift from a tightly coupled, polling-based model to a loosely coupled, event-driven one [64](#) [68](#). The cache allows different threads, such as those spawned by `:connect()` calls, to store and retrieve state information that the main loop can then act upon [62](#). This mechanism solves the primary challenge identified earlier: the inability of event handlers to communicate their status or results back to the central

control logic. By using string keys to store boolean flags, status messages, and numerical data, the system can create a shared, real-time source of truth accessible by all components.

For example, instead of the event handler for `market.changedSeedStock` simply executing `farming[3]()`, it can now orchestrate the entire background task and write its conclusion to the cache upon completion. A typical workflow would involve the handler first setting a flag like `cache.set("seeds_buying_in_progress", "true")`. Once the seed purchasing is finished, it would clear this flag and set a success message along with relevant data, such as `cache.set("last_market_action", "SEEDS_BOUGHT")` and `cache.set("action_details", "Pomegranate seeds acquired.")` ⁷⁶. The main loop in `Automation5.lau` would then continuously monitor these cache keys rather than relying on a fixed timer or a busy-wait loop. When it detects that `seeds_buying_in_progress` is false, it knows the background task is complete and can safely resume normal farming operations without any idle wait. This decoupling of long-running, non-critical tasks from the drone's primary control path is essential for minimizing execution time. The cache acts as the bridge that allows the system to react to events and state changes in near real-time, aligning with modern principles of event-driven processing found in various computing domains ^{18 61}.

The effectiveness of this approach hinges on careful key design and a disciplined locking strategy to prevent race conditions. Given that the cache only supports up to five string keys, strategic selection is paramount ⁴³. Keys should be designed to represent high-level states and outcomes (e.g., `CURRENT_STATE`, `MARKET_ACTION_STATUS`, `NEXT_TASK_TARGET`). To prevent conflicting writes, especially from multiple concurrent handlers, a mutual exclusion pattern must be implemented. The existing `if buyingSeeds then return end` logic in the `buySeedsOnRefresh` function is a rudimentary form of this, acting as a semaphore to ensure a task does not start while another instance is already running ¹¹². This pattern should be formalized across all event handlers that modify shared state. For instance, before starting a gear-buying operation, a handler would first check for a `gear_buying_in_progress` key. If it doesn't exist, it sets the key, performs its task, and removes the key upon completion. This ensures atomicity and data integrity, allowing the main loop to rely on the cache as a consistent and accurate reflection of the system's status. By leveraging the cache as the central nervous system of the application, the system can evolve from a slow, sequential processor into a responsive, parallel-aware agent capable of handling background tasks efficiently while keeping the drone actively engaged in productive work.

Architectural Transformation to a Reactive State Machine

The most impactful optimization involves a fundamental architectural shift from the current procedural main loop to a reactive state machine driven by the Cache library. This transformation directly addresses the primary bottleneck of artificial delays caused by the `task.wait(0.1)` call in `Automation5.lau`. Instead of iterating on a fixed schedule, the new main loop will respond to changes in the system's state as signaled through the cache. This approach minimizes idle time by ensuring the drone is always working towards the next highest-priority task dictated by the application's logic. The first step is to refactor the main loop to eliminate the fixed wait. The `task.wait(0.1)` at the bottom of the `while true` loop should be replaced with a much shorter, context-sensitive wait or removed entirely, allowing the loop to execute as quickly as possible to react to state changes ⁹³. The logic inside the loop will revolve around reading a `current_state` variable from the cache and dispatching to the appropriate handler function.

A proposed implementation would introduce a central state variable managed in the cache, such as `cache.set("app_state", "IDLE")`. The main loop would then follow a simple decision-making process: 1. Read the current state from the cache. 2. Based on the state, invoke the corresponding function from the `farming` or `movement` modules (e.g., if `PLANTING`, call `farming.sweep()`; if `HARVESTING`, call `farming.harvestPass()`). 3. After the function completes its logical block, update the cache with the next intended state (e.g., `cache.set("app_state", "CHECKING_MARKET")`).

This structure transforms the drone's behavior from being dictated by time to being dictated by events and state transitions. For instance, after completing a `sweep` operation, the state machine transitions to a harvesting phase. The event handlers for market refreshes would also participate in this state machine. Upon detecting a market refresh, the handler would not just execute a function; it would write a signal to the cache, such as `cache.set("pending_background_task", "BUY_SEEDS")`. The main loop, upon reading this pending task, could then transition to a state where it waits for the drone to finish its current movement, and once free, it can either execute the task itself if it's lightweight or, more likely, set a flag indicating that background buying is active and then immediately transition back to its primary farming state. The main loop would then poll the `background_task_active` flag and, upon seeing it turn false, proceed with its next action. This decouples the expensive market interactions from the drone's movement timeline, allowing the drone to continue its work while seeds and

gears are being purchased in the background. This event-driven paradigm is widely recognized for building systems that are responsive and scalable, reacting to streams of events rather than operating in batch mode [37](#) [61](#). By adopting this model, the system becomes inherently more efficient, reducing idle time and increasing the overall throughput of farming operations.

Action Sequence and Movement Optimization

Optimizing the sequence of actions and the drone's movement is crucial for minimizing in-game execution time. The current implementation relies heavily on blocking calls and inefficient traversal patterns. A key area for improvement is the refactoring of the `moveTo` function in `Movement.laum`. Currently, it contains a busy-wait loop that consumes CPU cycles while waiting for the drone to arrive at its destination [78](#). This can be eliminated by making the function truly asynchronous. The revised `moveTo` function would simply issue the `droneV2.goto(tx, tz)` command and return immediately, without waiting for the drone to stop. To track the drone's status, the main loop can use the cache to store the last target coordinates and an arrival status. For example, upon issuing a move command, the main loop would set `cache.set("last_move_target_x", tx)` and `cache.set("last_move_target_z", tz)`. The arrival status could be managed by a separate mechanism, perhaps triggered by a recurring, very short `task.wait()` in the main loop that checks the drone's position against the cached target. Once they match, the arrival status is updated in the cache to "ARRIVED". This change frees up the main processor during flight time, allowing it to plan the next move or check for other events, thus eliminating a major source of artificial delay.

The traversal logic within `FarmingOperations.laum` can also be optimized. The `sweep` function currently follows a fixed back-and-forth pattern, but it could be enhanced to prioritize areas of the field based on immediate needs, such as harvesting ready plants or planting seeds. By integrating with the reactive state machine, the main loop could instruct the drone to first execute a targeted `harvestPass` to collect ripe fruits before performing a general `plantPass` to replenish empty slots. This prioritization ensures that high-value assets (fruits) are collected promptly, and planting resources (seeds) are used efficiently. The `doTile` function, which handles the logic for a single grid cell, is central to this process. Its logic should be streamlined to minimize redundant checks. For instance, instead of repeatedly calling `drone.getPlant()` and checking its properties, these values can be stored in local variables to avoid repeated API

calls within the loop. Furthermore, the system should leverage the drone's ability to hover over a location to gather information without planting or harvesting, allowing it to build a more complete mental model of the field before committing to an action.

Another significant optimization opportunity lies in reducing module overhead and consolidating logic where feasible. The system is constrained by a 5-file limit and a 5200-character limit per file, which necessitates an efficient use of space [40](#). Functions in `Commands.laum` and `FarmingOperations.laum` that are frequently called and reference common modules like `Config.laum` could potentially be merged into a single module to reduce the number of `req()` calls and simplify the dependency graph. For example, helper functions related to inventory management or seed validation could be moved from `Commands.laum` into `FarmingOperations.laum`. This consolidation saves valuable character count and reduces the overhead of inter-module communication, allowing the code to run slightly faster and fit within the strict size constraints. Every character saved through refactoring can be reinvested into more sophisticated logic for crop selection or movement planning, directly contributing to the goals of increased speed and profitability. By systematically eliminating blocking waits, streamlining action logic, and optimizing the module structure, the drone's operational efficiency can be substantially improved.

Profitability Maximization Through Intelligent Sequencing

While minimizing execution time is the primary objective, maximizing profitability is a secondary but equally important goal. This can be achieved by moving beyond a static farming strategy and implementing an intelligent sequencing mechanism that prioritizes high-efficiency crops. The provided fruit value table offers a clear metric for profitability, with metrics like "Avg. Profit/min" providing a direct measure of return on investment [115](#) [122](#). The current system uses a static `preferredSeed` defined in `Config.laum`, typically set to `Enum.Seed.Pomegranate` [100](#). This approach is suboptimal because it ignores fluctuations in the market, such as the availability of seeds and the value of different fruits. A more dynamic system would allow the drone to adapt its strategy based on real-time data.

The `Cache` library is again the key enabler for this intelligence. The main loop can be programmed to read from the cache to determine the current profit-maximizing strategy.

For example, a special key like `cache.get("target_crop_profit_per_min")` could be used to identify the most profitable fruit available. When a chat command like `setseed Dragon` is received, instead of immediately changing the seed, the command handler would write this preference to the cache (e.g., `cache.set("next_seed_preference", Enum.Seed.Dragon)`). The main loop would then check this preference at the beginning of each farming cycle. If a preferred seed is available in the inventory and there are empty plots, it can prioritize planting that seed over others. The `plantPass` function in `FarmingOperations.laum` can be modified to accept a target seed type, allowing it to selectively plant only the most profitable variety when instructed to do so. This prevents the drone from wasting time and seeds on less valuable crops when higher-value options are available.

Furthermore, the harvesting sequence can be optimized. The `harvestPass` function already has logic to wait for plants to reach maturity, but it traverses the field in a predetermined order [124](#). The intelligent sequencing logic can enhance this by creating a priority queue of plants to harvest. As the drone scans the field, it can identify all ready-to-harvest fruits, calculate their expected profit based on their type, and sort them in descending order of "Avg. Profit/min". The drone would then traverse the field not in a simple back-and-forth pattern, but by visiting the coordinates of the highest-priority fruits first. This ensures that the most valuable assets are harvested and sold as quickly as possible, improving cash flow and allowing for faster replanting. The chat command `"status"` can be enhanced to display not just the count of empty, planted, and ready plants, but also the calculated total value of the ready fruits, giving the player insight into the system's profitability [42](#). By combining dynamic seed selection, targeted planting, and prioritized harvesting, the drone automation system can evolve from a simple repetitive task executor into a sophisticated economic agent that actively seeks to maximize its output, directly achieving the dual goals of speed and profitability.

Implementation Strategy and Risk Mitigation

The successful implementation of these optimizations requires a phased approach that respects the project's constraints and mitigates potential risks. The primary constraint is the 5-file, 5200-character-per-file limit, which demands a conservative yet effective refactoring strategy. The initial step should be to consolidate overlapping functionality. For instance, `Commands.laum` and `FarmingOperations.laum` both contain utility functions related to seeds and inventory. Merging these into a single module can reduce the number of required modules, saving character count and simplifying the codebase.

This consolidation should be done carefully, ensuring that function names are unique to avoid conflicts and that dependencies are clearly managed. Any new helper functions created for managing the Cache library (e.g., `cache.setFlag(key)`, `cache.getStatus(key)`) should be kept concise to minimize character usage.

The second phase involves the architectural transformation. The `Automation5.lau` file will undergo the most significant changes. The main loop must be rewritten as a reactive state machine, replacing the `task.wait(0.1)` with a non-blocking loop that polls the cache. All event handlers (`buySeedsOnRefresh`, `buyGearsOnRefresh`) must be refactored to act as self-contained workers that use the cache to report their status and results. This is a critical change that requires meticulous attention to detail to ensure all shared state is properly synchronized. The third phase focuses on optimizing individual functions. The `moveTo` function in `Movement.laum` must be converted to an asynchronous call, and the `doTile` function in `FarmingOperations.laum` should be streamlined to reduce redundant API calls. Finally, the logic for dynamic seed selection and prioritized harvesting will be integrated into the main loop and `FarmingOperations.laum`.

Several risks must be proactively managed. The foremost risk is exceeding the character limits. Before committing to a merge, the approximate character counts of each file should be estimated to ensure compliance. If a file approaches the 5200-character limit, the solution is to further optimize by shortening variable names (where clarity is not compromised) or by splitting a large function into smaller ones. Another risk is cache reliability and race conditions. Since the cache is volatile and has a small capacity, it must be used judiciously ⁷⁵. The locking mechanism using flags like `buyingSeeds` must be consistently applied to all shared resources in the cache to prevent data corruption. Empirical testing is essential to validate the system's behavior under load. A final risk is the assumption about API latencies. The actual time taken for `droneV2.goto()` and the precision of `drone.status()` updates are unknown ⁹⁷. The effectiveness of removing busy-wait loops depends on these timings. Therefore, iterative testing and measurement are crucial. By monitoring the drone's performance and the time taken for each cycle, the team can fine-tune the `task.wait()` durations in the main loop, if any are retained, to find the optimal balance between responsiveness and resource consumption. By following this structured implementation plan and actively mitigating these risks, the proposed optimizations can be successfully deployed, resulting in a highly efficient and profitable drone farming automation system.

Reference

1. Survey of Drones for Agriculture Automation from Planting to Harvest https://www.researchgate.net/publication/326040397_Survey_of_Drones_for_Agriculture_Automation_from_Planting_to_Harvest
2. [PDF] UAV search path planning for cattle monitoring: From linear ... <https://theses.hal.science/tel-05419244/document>
3. A Comprehensive Review of LiDAR Applications in Crop ... - MDPI <https://www.mdpi.com/1424-8220/24/16/5409>
4. UAVs Meet Agentic AI: A Multidomain Survey of Autonomous Aerial ... <https://arxiv.org/html/2506.08045v1>
5. Operations & supply chain management: principles and practice <https://www.tandfonline.com/doi/full/10.1080/00207543.2025.2555531>
6. QNX Inter-Process Communication Guide | PDF - Scribd <https://www.scribd.com/document/983665799/06-nto-ipc-r51>
7. ipc <https://www.cs.cmu.edu/~./IPC/>
8. [PDF] SYS/BIOS Inter-Processor Communication (IPC) 1.25 <https://www.ti.com/lit/pdf/sprug06>
9. Interprocess Communication Software https://docs.oracle.com/cd/E19282-01/E20948-01/chap5_IPC.html
10. Roblox Lua Coding Guide in 24 Hours | PDF | Control Flow - Scribd <https://www.scribd.com/document/797328012/Coding-With-Roblox-Lua-in-24-Hours-the-Official-Roblox-Roblox-Corporation-San-Mateo-California-Author-2021-Hoboken-Pearson-Education>
11. Computer Science Feb 2025 - arXiv <https://www.arxiv.org/list/cs/2025-02?skip=8900&show=2000>
12. FPT Camera - App Store - Apple <https://apps.apple.com/vn/app/fpt-camera/id1476239110>
13. Grady-White | Best Quality Center Console, Dual Console, & Cabin ... <https://www.gradywhite.com/>
14. [PDF] Computer Organization and Architecture by William Stallings http://home.ustc.edu.cn/~louwenqi/reference_books_tools/

Computer%20Organization%20and%20Architecture%2010th%20-%20William%20Stallings.pdf

15. Computer System - an overview | ScienceDirect Topics <https://www.sciencedirect.com/topics/computer-science/computer-system>
16. 1 Introduction - arXiv <https://arxiv.org/html/2411.18845v2>
17. A Review of Last-Mile Delivery Optimization - MDPI <https://www.mdpi.com/2504-446X/9/3/158>
18. Edge AI on Factory Floor: Real-Time Analytics & Predictive ... https://www.linkedin.com/posts/association-for-advancing-automation_edgeai-industrialautomation-activity-7440109884991266817-UNZM
19. Optimization and performance analysis of Drones and Unmanned ... <https://www.sciencedirect.com/science/article/pii/S2666307425000440>
20. Event-Driven Offloading and Load-Balancing With Quantized ... https://www.researchgate.net/publication/383765259_Event-Driven_Offloading_and_Load-Balancing_With_Quantized_Control_for_Fog-Enabled_UAVs
21. Energy-efficient neuromorphic computing for ultra-low latency ... <https://link.springer.com/article/10.1007/s44163-026-01093-7>
22. 333333 23135851162 the 13151942776 of 12997637966 <ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/words-333333.txt>
23. Experts for " Lp Ray Hindi - Linknovate <https://www.linknovate.com/search/?query=%E2%80%9D+lp+ray+hindi>
24. XBRL Viewer - SEC.gov https://www.sec.gov/ix?doc=/Archives/edgar/data/1842279/000121390022024612/fs42022a3_arclight2.htm
25. A Systematic Review of Serious Games in the Era of Artificial ... - MDPI <https://www.mdpi.com/2079-9292/14/4/649>
26. [PDF] Emerging Giants in Asia Pacific - KPMG agentic corporate services <https://assets.kpmg.com/content/dam/kpmg/xx/pdf/2022/07/emerging-giants-in-asia-pacific.pdf>
27. Plantation Monitoring Using Drone Images: A Dataset and ... - arXiv <https://arxiv.org/html/2502.08233v1>
28. Scientific Evolution and Core Bottleneck Analysis in the Field of UAV ... <https://www.mdpi.com/2076-3417/15/15/8304>
29. Invasive plants detection and distribution patterns analysis through ... <https://www.sciencedirect.com/science/article/abs/pii/S016816992401202X>
30. (PDF) Drones, Deep Learning, and Endangered Plants: A Method ... <https://www.researchgate.net/publication/>

367827298_Drones_Deep_Learning_and_Endangered_Plants_A_Method_for_Population-Level_Census_Using_Image_Analysis

31. CropQuant-Air: an AI-powered system to enable phenotypic analysis ... <https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2023.1219983/full>
32. Machine Learning Applications in Internet-of-Drones: Systematic ... <https://dl.acm.org/doi/10.1145/3571728>
33. Detecting plant species in the field with deep learning and drone ... <https://besjournals.onlinelibrary.wiley.com/doi/10.1111/2041-210X.13473>
34. Plant simulation software | Siemens <https://www.siemens.com/en-us/products/tecnomatix/plant-simulation-software/>
35. Is Lua an event-based programming language? <https://stackoverflow.com/questions/6192883/is-lua-an-event-based-programming-language>
36. Code Agent Sample: Lua Dungeon Crawler <https://docs.nvidia.com/nvigi-sdk/1.6.0/docs/SamplesCodeAgentLua.html>
37. [PDF] Structure- and Event-Driven Frameworks for State Machine ... - arXiv <https://arxiv.org/pdf/2604.00275>
38. LCM: LLM-focused Hybrid SPM-cache Architecture with Cache ... <https://dl.acm.org/doi/fullHtml/10.1145/3650200.3656592>
39. Master Luau Scripting for Roblox | PDF | Control Flow - Scribd <https://www.scribd.com/document/961880439/Mastering-Luau>
40. ChatGPT projects - 20 file limit - OpenAI Developer Community <https://community.openai.com/t/chatgpt-projects-20-file-limit/1064482>
41. Automate Farming with The Farmer Was Replaced <https://www.tiktok.com/@metaroot.ch/video/7557694935210691862>
42. [PDF] Treasure trail guide osrs 2 factor guide - Weebly <https://movefamunumuxe.weebly.com/uploads/1/4/1/9/141968622/zoberetimifid.pdf>
43. Simple Index - SUSTech Open Source Mirrors <https://mirrors.sustech.edu.cn/pypi/simple/>
44. [PDF] Modern System-on-Chip Design on Arm - Microsoft .NET <https://armkeil.blob.core.windows.net/developer/Files/pdf/ebook/arm-modern-soc-design-on-arm.pdf>
45. Scripting API: EventSystem - Unity - Manual <https://docs.unity3d.com/2018.2/Documentation/ScriptReference/EventSystems.EventSystem.html>
46. Class EventSystem | Unity UI | 1.0.0 <https://docs.unity.cn/Packages/com.unity.ugui@1.0/api/UnityEngine.EventSystems.EventSystem.html>

47. How do you reference UI Event System from script? - Stack Overflow <https://stackoverflow.com/questions/30948274/how-do-you-reference-ui-event-system-from-script>
48. Event Management - Developer Documentation <https://developer.siemens.com/insights-hub/docs/apis/advanced-eventmanagement/api-eventmanagement-overview.html>
49. XBRL Viewer - SEC.gov <https://www.sec.gov/ix?doc=/Archives/edgar/data/1819142/000110465921152265/ivan-20211221xs4a.htm>
50. Java Troubleshooting Guide: Release 23 | PDF - Scribd <https://www.scribd.com/document/867589173/Troubleshooting-Guide>
51. 3 - CodaLab Worksheets <https://worksheets.codalab.org/rest/bundles/0xd74f36104e7244e8ad99022123e78884/contents/blob/frequent-classes>
52. Middle States Commission on Higher Education | LinkedIn <https://www.linkedin.com/company/mscheorg>
53. Technology Solutions for Restaurants and Venues - Oracle <https://www.oracle.com/food-beverage/>
54. Computer Science - arXiv <https://arxiv.org/list/cs/new>
55. How to import external library in Roblox script - lua - Stack Overflow <https://stackoverflow.com/questions/73857012/how-to-import-external-library-in-roblox-script>
56. Automate Farming: Program a Drone for Maximum Efficiency <https://www.tiktok.com/@metaroot.ch/video/7552533366218804502>
57. Scalable Merged Pumpkin Farm (Full Automation Guide) <https://steamcommunity.com/sharedfiles/filedetails/?id=3597605446>
58. [PDF] Performance and Best Practices Guide for IBM Storage FlashSystem ... <https://www.redbooks.ibm.com/redbooks/pdfs/sg248543.pdf>
59. Fast Persistent Heap Based on Non-Volatile Memory - ResearchGate https://www.researchgate.net/publication/316602809_Fast_Persistent_Heap_Based_on_Non-Volatile_Memory
60. SrFTL: Leveraging Storage Semantics for Effective Ransomware ... <https://dl.acm.org/doi/full/10.1145/3767322>
61. Event-Driven Programming - 05 (Pre-final) Flashcards - Quizlet <https://quizlet.com/ph/824364814/event-driven-programming-05-pre-final-flash-cards/>
62. Networking and Interprocess Communication — Python 3.14.5 ... <https://docs.python.org/3/library/ipc.html>
63. Introduction — eventsourcing 9.5.5 documentation - Read the Docs <https://eventsourcing.readthedocs.io/en/stable/topics/introduction.html>

64. Interprocess Communications - Oracle Solaris Trusted Extensions ... https://docs.oracle.com/cd/E23823_01/html/819-0869/ipcapi-1.html
65. Why does ram memory increase in idle stage? - Stack Overflow <https://stackoverflow.com/questions/78864224/why-does-ram-memory-increase-in-idle-stage>
66. Subject-Oriented Business Process Management - Springer Nature <https://link.springer.com/content/pdf/10.1007/978-3-031-40213-5.pdf>
67. ipc - Softwaredevelopment patterns for Files and Caches as Inter ... <https://stackoverflow.com/questions/10736396/softwaredevelopment-patterns-for-files-and-caches-as-inter-process-communication>
68. The Interprocess Communication (IPC) Overview - IBM <https://www.ibm.com/support/pages/interprocess-communication-ipc-overview>
69. [PDF] System Software for M7–300/400 Program Design - Support https://support.industry.siemens.com/cs/attachments/1136533/M7phb_e.pdf
70. [PDF] Operating system dedicated to NVRAM-based low power embedded ... <https://theses.hal.science/tel-03192646v1/file/thesis.pdf>
71. [PDF] netX Insiders Guide <http://fs.gongkong.com/files/technicalData/200806/2008061913124600001.pdf>
72. Interprocess communication in Lua with Example? - Stack Overflow <https://stackoverflow.com/questions/40893558/interprocess-communication-in-lua-with-example>
73. GitHub 热榜项目- 日榜(2026-03-25) <https://zhuanlan.zhihu.com/p/2020072532267520319>
74. Roblox Lua Scripting Basics Guide | PDF | Boolean Data Type - Scribd <https://www.scribd.com/document/796317865/Detailed-Python-Roblox-Lua-Tutorials>
75. <https://developer.work.weixin.qq.com/devtool/intro...> <https://developer.work.weixin.qq.com/devtool/introduce?id=36391>
76. Public App Purchase - 服务端 API - Documentation - Lark Developer <https://open.larksuite.com/document/uAjLw4CM/ukTMukTMukTM/application-v6/event/public-app-purchase>
77. Redmart api interface guide - 淘宝开放平台 <https://jaq-doc.alibaba.com/docs/doc.htm?treeId=499&articleId=121567&docType=1>
78. [PDF] μ C/OS-III Users Manual - Analog Devices <https://www.analog.com/media/en/dsp-documentation/software-manuals/micrium-ucos-iii-usersmanual.pdf>
79. Roblox Studio: Game Creation Guide | PDF - Scribd <https://www.scribd.com/document/622534941/Lab-Module-1>
80. XBRL Viewer - SEC.gov <https://www.sec.gov/ix?doc=/Archives/edgar/data/0001342874/000134287424000018/tx-20231231.htm>

81. Guide :: All codes I used - Steam Community <https://steamcommunity.com/sharedfiles/filedetails/?id=3586817011>
82. XBRL Viewer - SEC.gov https://www.sec.gov/ix?doc=/Archives/edgar/data/1848275/000101376224002682/ea0210143-20f_topfinan.htm
83. [PDF] Modern C - HAL-Inria <https://inria.hal.science/hal-02383654v2/file/modernC.pdf>
84. [XLS] Occupations - ESCO https://esco.ec.europa.eu/system/files/2023-09/ESCO_v1.2%20consultation%20content%20overview.xlsx
85. [PDF] HiPEAC Vision 2023 - HAL-Inria https://inria.hal.science/hal-04023794/file/HiPEAC_Vision_2023.pdf
86. XBRL Viewer - SEC.gov <https://www.sec.gov/ix?doc=/Archives/edgar/data/0001030894/000103089421000013/cls-20201231.htm>
87. Concurrent version of LRU cache - Stack Overflow <https://stackoverflow.com/questions/39615187/concurrent-version-of-lru-cache>
88. Global Economic Strategies Amid Trade War | PDF - Scribd <https://www.scribd.com/document/886597328/TheWallStreetJournal-04-12-2025-w>
89. lua generic for-loop specification - Stack Overflow <https://stackoverflow.com/questions/34190533/lua-generic-for-loop-specification>
90. Programming in NodeMCU <https://nodemcu.readthedocs.io/en/dev-esp32/nodemcu-pil/>
91. Strictly Typed Functional Programming for Roblox - DEV Community <https://dev.to/jesterxl/strictly-typed-functional-programming-for-roblox-2bcn>
92. XBRL Viewer - SEC.gov <http://www.sec.gov/ix?doc=/Archives/edgar/data/1819142/000110465921152265/ivan-20211221xs4a.htm>
93. [PDF] Latency-aware and -predictable Communication with Open Protocol ... <https://arxiv.org/pdf/2010.09584>
94. On-board Real-time Event-image Fusion SLAM System for Drones <https://dl.acm.org/doi/full/10.1145/3696420>
95. AI-driven drone technology and computer vision for early detection ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC12820128/>
96. Real-Time Feasibility of Digital Twins for Process Control - MDPI <https://www.mdpi.com/2075-1702/14/6/595>
97. (PDF) SHYANNE ROSHAN Real-Time Control Loop Timing and ... https://www.researchgate.net/publication/401761285_SHYANNE_ROSHAN_Real-Time_Control_Loop_Timing_and_Latency_in_Autonomous_Robotics
98. Low-level memory and timing analysis of flight code for unmanned ... <https://link.springer.com/article/10.1007/s42401-024-00279-0>

99. The BEST Fruit Value Tier List in Blox Fruits! - #bloxfruits <https://www.tiktok.com/@vjay.roblox/video/7308041740617846058>
100. WORST to BEST MUTATIONS in Grow a Garden! <https://www.tiktok.com/@ayzegamesofficial/video/7506567354831326510>
101. What Does A Starfruit Make Per Second Plant Vs Brainrot <https://www.tiktok.com/discover/what-does-a-starfruit-make-per-second-plant-vs-brainrot>
102. Star Fruit Value Grow A Garden <https://www.tiktok.com/discover/star-fruit-value-grow-a-garden>
103. Discover the Most Expensive Starfruit Varieties <https://www.tiktok.com/@tt3xo00/video/7505595495956368671>
104. [XML] <https://www.ericsson.com/Sitemap.xml> <https://www.ericsson.com/Sitemap.xml>
105. [PDF] csi-logistics-trend-radar-6-dhl.pdf <https://www.dhl.com/content/dam/dhl/global/csi/documents/pdf/csi-logistics-trend-radar-6-dhl.pdf>
106. Sustainability, Volume 17, Issue 21 (November-1 2025) – 563 articles <https://www.mdpi.com/2071-1050/17/21>
107. [PDF] Technology Trends Outlook 2024 - McKinsey <https://www.mckinsey.com/~/media/mckinsey/business%20functions/mckinsey%20digital/our%20insights/the%20top%20trends%20in%20tech%202024/mckinsey-technology-trends-outlook-2024.pdf>
108. [PDF] glo-csi-logistics-trend-radar-7-0.pdf - DHL <https://www.dhl.com/content/dam/dhl/global/csi/documents/pdf/glo-csi-logistics-trend-radar-7-0.pdf>
109. [PDF] Technology Trends Outlook 2023 - McKinsey <https://www.mckinsey.com/~/media/mckinsey/business%20functions/mckinsey%20digital/our%20insights/mckinsey%20technology%20trends%20outlook%202023/mckinsey-technology-trends-outlook-2023-v5.pdf>
110. Silver Prices Surge Amid Market Trends | PDF - Scribd <https://www.scribd.com/document/986465690/BL-Delhi-15-01-2026>
111. Embedded - TopPodcast.com https://toppodcast.com/podcast_feeds/embedded-2/
112. lru-cache - NPM <https://www.npmjs.com/package/lru-cache>
113. Blox Fruits Wiki | Fandom https://blox-fruits.fandom.com/wiki/Blox_Fruits
114. Full Fruit Tier List: Green Apples Ranked <https://www.tiktok.com/@sillz/video/7476836382527196447>
115. Grow A Garden Fruit Value List | TikTok <https://www.tiktok.com/discover/grow-a-garden-fruit-value-list>
116. [PDF] The next big arenas of competition - McKinsey Global Institute <https://www.mckinsey.com/~/media/mckinsey/mckinsey%20global%20institute/>

our%20research/the%20next%20big%20arenas%20of%20competition/the-next-big-arenas-of-competition_final.pdf

117. Metaverse and Digital Twins in the Age of AI and Extended Reality <https://www.mdpi.com/2673-8945/5/2/36>
118. Speculative Key-Value Caching for Efficient Generation of LLMs <https://arxiv.org/html/2503.16163v1>
119. Erlang style concurrency in the D programming language <https://stackoverflow.com/questions/4011860/erlang-style-concurrency-in-the-d-programming-language>
120. [PDF] Some PL Features: Lua - 中国科学技术大学 <http://staff.ustc.edu.cn/~yuzhang/pldpa/notes/A3lua.pdf>
121. How to do the scratch test to see if your tree is alive or dead #garde... <https://www.tiktok.com/@andrethefarmer/video/7613735449235901709>
122. Computer Practice Farming N5 June 2026 Questions Paper - TikTok <https://www.tiktok.com/content/computer-practice-farming-n5-june-2026-questions-paper>
123. Walkthrough - IGN <https://www.ign.com/articles/2009/10/21/scribblenauts-walkthrough-1026310>
124. Grow a Garden in 24 Hours on Roblox: Part 1 <https://www.tiktok.com/@gopeachy/video/7517718855817465119>